Instituto Politécnico Nacional

Escuela Superior de Cómputo - IPN

ESCOM

Teoría de la computación

Act. 8, Ejercicio 2 - Diseño de expresión regular y autómata finito

Profesor:

Sánchez Garcia Luz María

Alumnos:

- García Ambrosio Aldo
- Leaños Gutierrez Alan Rodrigo
- Pérez Marquez David Andrawi

Equipo: Red One

Grupo:

4CM5

Fecha de entrega:

Miércoles 25 de marzo de 2026

Introducción

En el estudio de la Teoría de la Computación, los autómatas finitos constituyen uno de los modelos matemáticos fundamentales para el reconocimiento y validación de lenguajes formales. Un autómata finito es una máquina abstracta compuesta por un conjunto finito de estados, un alfabeto de entrada, una función de transición, un estado inicial y uno o más estados de aceptación. Su capacidad para describir patrones estructurados lo convierte en una herramienta indispensable en áreas como la compilación, el procesamiento de texto y la validación de datos.

Estrechamente vinculadas a los autómatas finitos, las expresiones regulares (ER) son notaciones algebraicas que describen de manera concisa los lenguajes regulares, es decir, aquellos que pueden ser reconocidos por un autómata finito determinista (AFD) o no determinista (AFN). Existe una equivalencia formal entre ambos formalismos, demostrada por el Teorema de Kleene, lo que permite construir un autómata a partir de cualquier expresión regular y viceversa.

Objetivo de la Práctica

La presente práctica tiene como finalidad aplicar los fundamentos teóricos de los lenguajes regulares y los autómatas finitos en un caso de uso concreto del mundo real: la validación de direcciones IP versión 4 (IPv4). Para ello, se hará uso del software JFlap (Java Formal Languages and Automata Package), una herramienta de simulación ampliamente utilizada en el área de la Teoría de la Computación que permite construir, visualizar y ejecutar autómatas de manera interactiva.

De manera específica, la práctica persigue los siguientes objetivos:

1. Diseñar la expresión regular que modele formalmente el lenguaje de las direcciones IPv4 válidas.
2. Construir el Autómata Finito No Determinista (AFN) equivalente a dicha expresión regular mediante el algoritmo de construcción de Thompson, obteniendo así un autómata con transiciones-ε.
3. Convertir el AFN al Autómata Finito Determinista (AFD) equivalente mediante el procedimiento de construcción de subconjuntos (subset construction), eliminando el no determinismo y las transiciones-ε.
4. Validar el comportamiento de ambos autómatas en JFlap, verificando la aceptación o rechazo de cadenas.

¿Qué es una Dirección IP?

Una dirección IP (Internet Protocol Address) es un identificador lógico, numérico y jerárquico asignado a cada interfaz de red de un dispositivo que participa en una red informática que utiliza el protocolo de Internet (IP) como base de comunicación. Este protocolo fue definido originalmente en el documento RFC 791 publicado en 1981 por DARPA, y constituye la base del modelo de comunicación en Internet.

La dirección IP cumple dos funciones esenciales dentro de una red:

- Id del host: Distingue de forma unívoca a un dispositivo dentro de una red.
- Direccionamiento y enrutamiento: Permite a los routers determinar la ruta óptima para el envío de paquetes de datos desde el origen hasta el destino.

Existen dos versiones del protocolo IP actualmente en uso:

Versión	Longitud	Representación	Espacio de direcciones
IPV4	32 bits	Decimal punteada	~4,294 millones de direcciones
IPV6	128 bits	Hexadecimal con colons	~3.4×10^{38} direcciones

Para efectos de esta práctica, se trabajará exclusivamente con IPv4, por ser el formato más representativo desde el punto de vista del diseño de autómatas, al presentar restricciones numéricas precisas y una estructura repetitiva que resulta ideal para su modelado formal.

Estructura de una Dirección IPv4

Una dirección IPv4 está compuesta por cuatro octetos, cada uno representando 8 bits de información, separados entre sí por un punto (.). Su forma general es:

$$A \,.\, B \,.\, C \,.\, D$$

Donde A, B, C y D son valores enteros decimales pertenecientes al rango [0, 255]. Esto significa que cada octeto puede asumir 256 valores posibles (2^8), dando lugar a un espacio total de 2^{32} = 4,294,967,296 direcciones únicas.

Restricciones por número de dígitos

Dado que cada octeto debe estar en el rango [0, 255], las cadenas válidas se clasifican según su longitud en dígitos:

Caso	Rango	Ejemplo
1 dígito	0 - 9	0, 7, 9,6
2 dígitos	10 - 99	10, 45, 99, 46
3 dígitos (1xx)	100 - 199	100, 172, 199, 138
3 dígitos (2xx)	200 - 249	200, 224, 249, 215
3 dígitos (25x)	250 - 255	250, 253, 255, 251

Valores como 256, 300 o 999 son inválidos y deben ser rechazados por el autómata.

Alfabeto y Lenguaje Formal

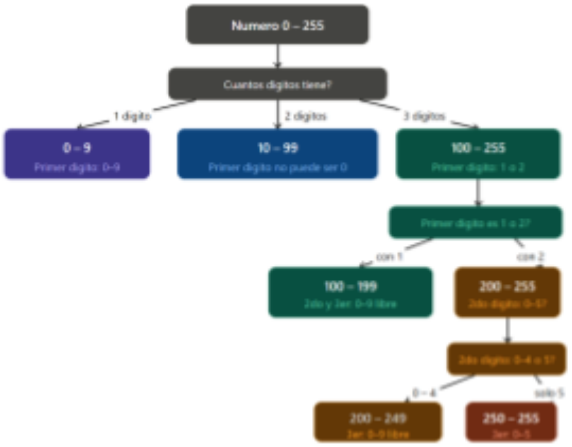
Desde la perspectiva de la Teoría de la Computación, una dirección IPv4 puede ser tratada como una cadena perteneciente a un lenguaje regular definido sobre el siguiente alfabeto:

$$\Sigma = \{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, . \}$$

El lenguaje formal que se desea reconocer es:

$$L(IPv4) = \{ w \in \Sigma^* \mid w \text{ es una dirección IPv4 válida} \}$$

Este lenguaje es regular, lo que garantiza la existencia de una expresión regular y un autómata finito (tanto AFN como AFD) capaz de reconocerlo.



Expresión Regular

El principal problema al diseñar la expresión regular radica en que los motores de expresiones operan sobre caracteres individuales y no sobre valores numéricos, por lo que no es posible expresar directamente una condición del tipo "número menor o igual a 255".

> *Para resolverlo, se analiza el rango 0–255 y se descompone en cinco subrangos mutuamente excluyentes, definidos según la cantidad de dígitos y el valor permitido en cada posición.*

Cada subrango se traduce a un patrón de caracteres independiente, y todos se combinan mediante el operador de alternación (|). El orden en que aparecen los casos dentro de la alternación es deliberado; los patrones más específicos se colocan primero para evitar que el motor acepte una coincidencia parcial antes de intentar la más completa.

- **Caso 1: 25[0-5] → cubre 250 a 255<**

  Los dos primeros caracteres son fijos y obligatoriamente 2 y 5. El tercer dígito solo puede ser 0, 1, 2, 3, 4 o 5, lo que se expresa con la clase de caracteres [0-5]. No puede ser mayor porque está fuera del rango permitido en una dirección IPv4.

- **Caso 2: 2[0-4][0-9] → cubre 200 a 249**

  El primer dígito es 2, el segundo puede ser 0, 1, 2, 3 o 4 (no 5, porque esos ya están cubiertos por el Caso 1), y el tercero puede ser cualquier dígito del 0 al 9.

- **Caso 3: 1[0-9][0-9] → cubre 100 a 199**

  El primer dígito es 1. Las posiciones 2 y 3 pueden ser cualquier dígito del 0 al 9, porque 100 hasta 199 nunca se salen del límite superior.

- **Caso 4: [1-9][0-9] → cubre 10 a 99**

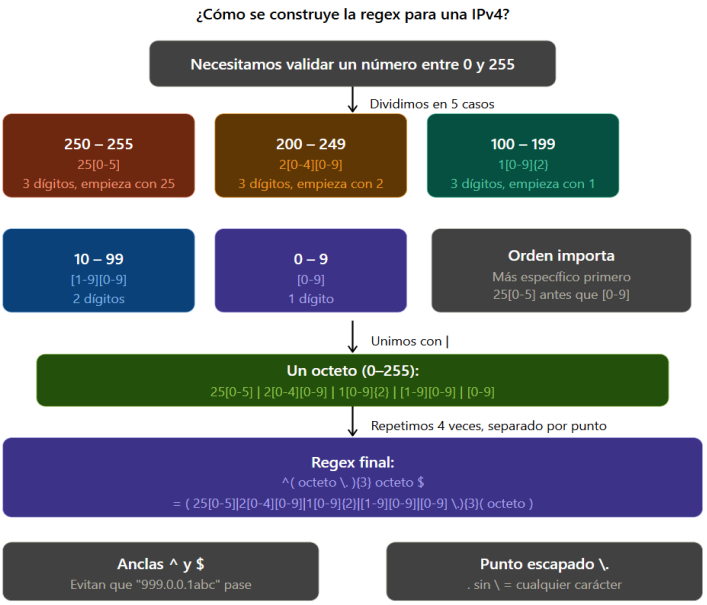
  Este caso maneja números de exactamente dos dígitos. El primer dígito va de 1 a 9 (no puede ser 0 porque generaría 01, 02, etc., que son representaciones no válidas en una IP). El segundo dígito puede ser cualquier cosa de 0 a 9.

- **Caso 5: [0-9] → cubre 0 a 9**

  El caso más simple: un único dígito del 0 al 9. Cubre los números de un solo dígito.

Una vez definidos los cinco casos que cubren el rango completo de un octeto, el siguiente paso es ensamblar la expresión regular que valide una dirección IPv4 completa.

Para ello, se agrupan los cinco patrones en una sola subexpresión mediante el operador de alternación (|), y esa subexpresión se repite cuatro veces con el punto literal como separador; el punto debe escribirse escapado como \. para que el motor lo interprete como un carácter literal y no como el metacarácter que representa cualquier carácter; y se añaden las anclas ^ y $ al inicio y al final de la expresión para garantizar que la cadena evaluada sea exclusivamente la dirección IP, sin caracteres adicionales antes ni después.



La expresión regular resultante es la siguiente:

^((25[0-5]|2[0-4][0-9]|1[0-9][0-9]|[1-9][0-9]|[0-9])\.){3}(25[0-5]|2[0-4][0-9]|1[0-9]{2}|[1-9][0-9]|[0-9])$

- La expresión regular puede verificarse en línea con casos de prueba representativos en el siguiente enlace: https://regex101.com/r/BkbrYa/1

Autómatas Finitos

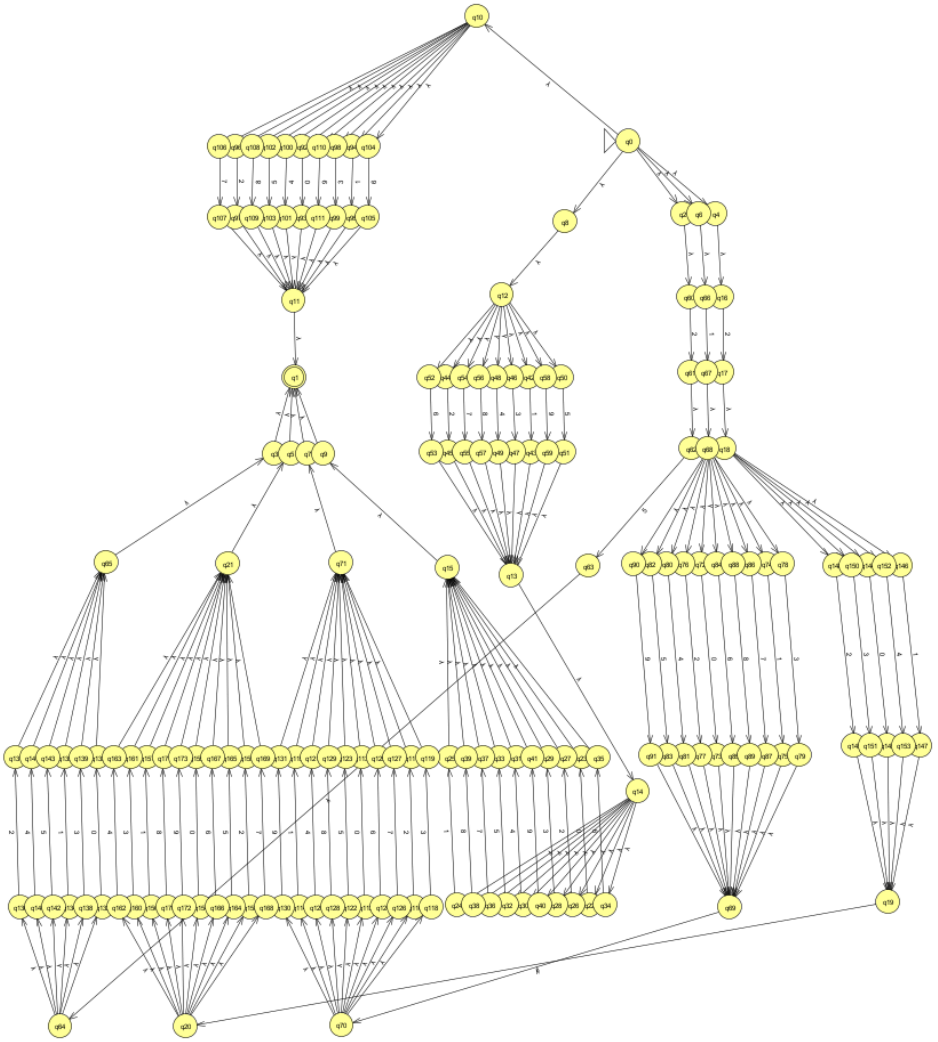
1.- AFN (Construcción de Thompson)

El algoritmo de construcción de Thompson (1968) es un método sistemático para convertir cualquier expresión regular en un Autómata Finito No Determinista con transiciones vacías (AFN-ε). El resultado es un autómata que:

- Tiene exactamente un estado inicial y un estado de aceptación.
- Utiliza transiciones-ε (épsilon) para conectar los subautómatas de cada subexpresión.
- Construye el autómata completo de forma composicional, aplicando reglas inductivas sobre la estructura de la expresión regular.

Estructura del AFN para IPv4

Dado que la expresión regular de IPv4 está construida sobre la alternación de 5 casos por octeto, el AFN de Thompson genera la siguiente arquitectura general:



Para la construcción del AFN se utilizó el software JFlap en su modalidad de Regular Expression. Dado que JFlap opera con su propio dialecto de expresiones regulares, fue necesario adaptar la expresión estándar al formato compatible con la herramienta, realizando las siguientes conversiones:

Notación estándar	**Notación JFlap**
Operador de alternación |	Operador +
Clases de caracteres [a-b]	Alternación explícita (a+...+b)
Cuantificador {n}	Repetición manual
Anclas ^ y $	No se utilizan
Punto escapado \.	Punto literal .

La expresión adaptada para un solo octeto válido (rango 0–255) ingresada en JFlap fue la siguiente:

(25(0+1+2+3+4+5)+2(0+1+2+3+4)(0+1+2+3+4+5+6+7+8+9)+1(0+1+2+3+4+5+6+7+8+9)(0+1+2+3+4+5+6+7+8+9)+(1+2+3+4+5+6+7+8+9)(0+1+2+3+4+5+6+7+8+9)+(0+1+2+3+4+5+6+7+8+9))

Esta expresión se introduce en JFlap seleccionando la opción Regular Expression en el menú principal, y posteriormente se ejecuta la conversión mediante ***Convert*** → ***Convert to NFA***, lo que desencadena la aplicación automática del algoritmo de Thompson.

Resultado: AFN Obtenido

El AFN generado por JFlap a partir de la construcción de Thompson presenta la estructura característica del algoritmo: una arquitectura en capas, donde cada nivel corresponde a una etapa de procesamiento de los dígitos del octeto.

Para verificar el correcto funcionamiento del autómata, se utilizaron las herramientas de simulación integradas en JFlap.

Prueba múltiple (Multiple Run)

Mediante la opción ***Input*** → ***Multiple Run*** se ejecutaron simultáneamente múltiples cadenas de prueba, obteniendo los siguientes resultados:

Cadena de prueba	Resultado obtenido	Resultado esperado
0	Accept	Accept
9	Accept	Accept
99	Accept	Accept
100	Accept	Accept
199	Accept	Accept
200	Accept	Accept
249	Accept	Accept
250	Accept	Accept
255	Accept	Accept
256	Reject	Reject
300	Reject	Reject
999	Reject	Reject

- La validación se realizó sobre la expresión del octeto individual como unidad base del autómata, puesto que la expresión regular completa de IPv4 (cuatro octetos separados por punto) genera un AFN de más de 600 estados, cuyo diagrama resulta visualmente inmanejable.

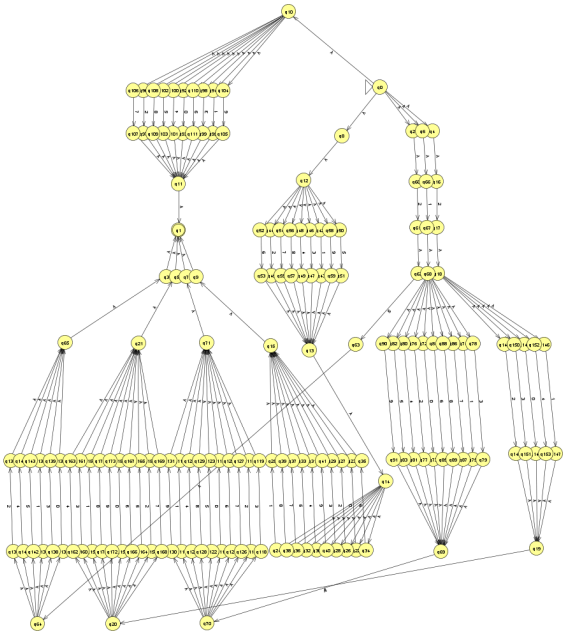


Table Text Size

Input	Result
0	Accept
9	Accept
99	Accept
100	Accept
199	Accept
200	Accept
249	Accept
250	Accept
255	Accept
256	Reject
300	Reject
999	Reject

Los resultados obtenidos son consistentes con el comportamiento esperado del lenguaje L(octeto) = { w ∈ Σ* | w representa un entero decimal en el rango [0, 255] }, confirmando que el AFN construido por Thompson reconoce correctamente el lenguaje definido por la expresión regular del octeto IPv4.

2.- AFD (Conversión por Construcción de Subconjuntos)

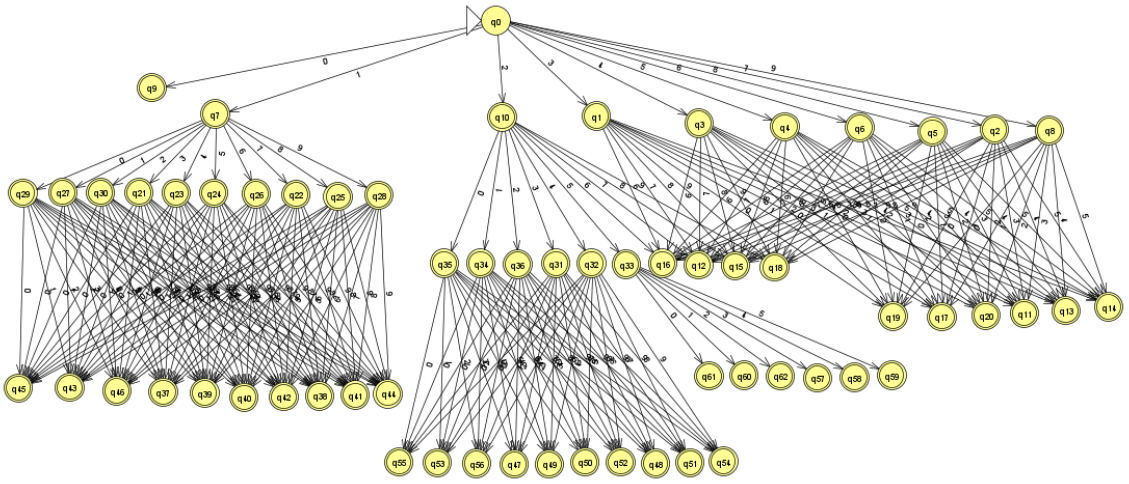
El procedimiento de construcción de subconjuntos (subset construction) permite convertir cualquier AFN-ε en un AFD equivalente. El principio central es que cada estado del AFD corresponde a un conjunto de estados del AFN que pueden ser activos simultáneamente al procesar un símbolo de entrada.

Una vez obtenido el AFN mediante el método de Thompson, el siguiente paso es la transformación a un Autómata Finito Determinista (AFD). Este proceso es fundamental porque, aunque el AFN es útil para el diseño basado en expresiones regulares, el AFD es el modelo que realmente se implementa en sistemas computacionales debido a su eficiencia: por cada estado y símbolo de entrada, existe una única transición posible.

Fundamento del Algoritmo

El procedimiento utilizado es la Construcción de Subconjuntos (*subset construction*), el cual se basa en agrupar estados del AFN que pueden estar activos al mismo tiempo tras procesar una cadena. Los conceptos clave aplicados fueron:

- **ε-clausura:** Identificación de todos los estados alcanzables desde un estado q utilizando únicamente transiciones vacías (ε).
- **Función de transición extendida:** Definición de los nuevos estados del AFD como conjuntos de estados del AFN original.
- **Determinismo estricto:** Eliminación de cualquier ambigüedad en la lectura de los dígitos del octeto.



Resultado de la Conversión en JFlap

Utilizando la herramienta ***Convert*** → ***Convert to DFA*** en JFlap, se obtuvo un autómata optimizado para el lenguaje de un octeto (0-255). A diferencia de la estructura "en capas" y ramificada del AFN de Thompson, el AFD presenta una arquitectura más lineal y compacta, lo que reduce la carga computacional en el reconocimiento de cadenas.

El alfabeto de entrada para este autómata se mantiene como el subconjunto numérico de Σ, enfocado en validar que la secuencia de caracteres no exceda el límite superior de 255 ni presente formatos inválidos.

Validación del Comportamiento

Para garantizar la equivalencia formal entre el AFN y el nuevo AFD, se ejecutó nuevamente una batería de pruebas de entrada. Los resultados confirman que el lenguaje permanece inalterado:

Cadena de prueba	Resultado obtenido	Resultado esperado
0	Accept	Accept
7	Accept	Accept
45	Accept	Accept
100	Accept	Accept
199	Accept	Accept
200	Accept	Accept
249	Accept	Accept
250	Accept	Accept
255	Accept	Accept
256	Reject	Reject

299	Reject	Reject
300	Reject	Reject

Como se observa en los resultados de la tabla anterior, el **AFD** mantiene una precisión del 100% en el reconocimiento del lenguaje L(octeto). La transición exitosa desde la expresión regular, pasando por el AFN de Thompson hasta llegar al AFD, demuestra que la descomposición del rango [0, 255] en subpatrones de caracteres fue la estrategia correcta para superar la limitación de los motores de búsqueda que no procesan valores numéricos de forma nativa.

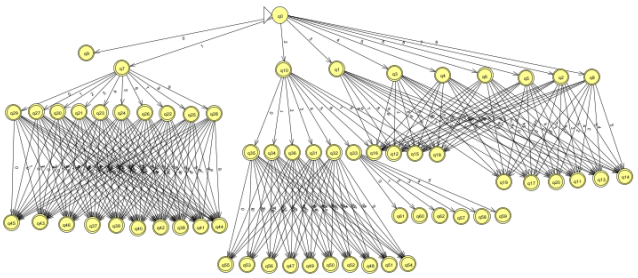


Table Text Size

Input	Result
0	Accept
7	Accept
45	Accept
100	Accept
199	Accept
200	Accept
249	Accept
250	Accept
255	Accept
256	Reject
299	Reject
300	Reject

Nota sobre la Escalabilidad del Autómata

Si bien el diseño presentado se centra en un solo octeto para facilitar su análisis y visualización, la construcción de la dirección **IPv4 completa** (octeto . octeto . octeto . octeto) sigue un modelo de concatenación secuencial.

Es importante notar que:

- **Crecimiento de estados:** El autómata completo de una IPv4 bajo el algoritmo de Thompson genera una estructura de más de 600 estados.
- **Jerarquía:** El octeto validado en esta sección funciona como un "módulo" o subrutina; si este componente es formalmente correcto, la integridad del sistema completo está garantizada mediante la operación de concatenación de lenguajes regulares.

Conclusión

La realización de esta práctica permitió comprobar los conceptos de teoría de la computación que tienen aplicación directa en problemas reales. Diseñar un validador de direcciones IPv4 implicó traducir una restricción del mundo real (un número entre 0 y 255, repetido cuatro veces) a un modelo formal ejecutable, lo cual no es trivial dado que los motores de expresiones regulares operan sobre caracteres y no sobre valores numéricos.

La solución, descomponer el rango en cinco subrangos mutuamente excluyentes, resultó ser la estrategia correcta y fue confirmada con 12 cadenas de prueba que cubrieron tanto los valores límite válidos como los casos de rechazo esperados, obteniendo resultados correctos en el 100% de los casos.

A lo largo del proceso también se hizo evidente la diferencia entre el AFN y el AFD: mientras el primero refleja fielmente la lógica de diseño, el segundo ofrece un modelo más eficiente e implementable en sistemas reales. Ambos reconocen exactamente el mismo lenguaje, lo que valida en la práctica el Teorema de Kleene.

Por último, el uso de JFlap como herramienta de simulación resultó fundamental para visualizar y verificar cada etapa del proceso. La posibilidad de construir autómatas a partir de expresiones regulares, convertirlos y ejecutar pruebas de forma interactiva facilitó la comprensión de conceptos que de otro modo podrían resultar puramente abstractos, consolidando así el vínculo entre la teoría formal y su aplicación práctica en el desarrollo de sistemas computacionales.